

講義 9：Functional ALV 與 MACRO

對應練習：ex09 | 答案程式：ZR_TR09_ALV

本講重點

- ALV 是什麼、為什麼取代純 WRITE 清單
- fieldcat (欄位目錄)：告訴 ALV 每個欄位怎麼顯示
- REUSE_ALV_GRID_DISPLAY 的呼叫方式與常用參數
- layout 版面設定 (斑馬紋、欄寬最佳化)
- MACRO (DEFINE ... END-OF-DEFINITION)：看得懂舊程式的必修

1. ALV 是什麼

ALV (ABAP List Viewer) 是 SAP 標準的表格顯示元件：使用者拿到的不是死板的文字清單，而是能自行排序、篩選、加總、調欄寬、匯出 Excel 的互動表格——這些功能全部內建，一行都不用寫。實務報表輸出九成用 ALV。

兩種寫法：

寫法	核心	定位
Functional ALV	CALL FUNCTION 'REUSE_ALV_GRID_DISPLAY'	傳統寫法，舊程式主流 (本講)
OO ALV	cl_salv_table / cl_gui_alv_grid	新程式建議，留待 OOP 課程

2. fieldcat：欄位目錄

ALV 需要知道「顯示哪些欄位、標題是什麼、多寬」——這份說明書就是 fieldcat，一欄一列：

```
DATA: gt_fieldcat TYPE slis_t_fieldcat_alv,      " fieldcat 內表
      gs_fieldcat TYPE slis_fieldcat_alv.        " 一欄的設定

CLEAR gs_fieldcat.
gs_fieldcat-fieldname = 'CARRID'.                " 資料內表的欄位名 ( 大寫！ )
gs_fieldcat-seltext_l = '航空公司代碼'.          " 欄位標題 ( long )
gs_fieldcat-outputlen = 12.                      " 顯示寬度
APPEND gs_fieldcat TO gt_fieldcat.
```

(slis_... 型別來自標準型別群組 SLIS，程式裡直接用即可。) 常用欄位：

fieldcat 欄位	用途
fieldname	對應資料內表的欄位名，必須大寫，打錯該欄直接不顯示
seltext_l / seltext_m / seltext_s	長 / 中 / 短欄位標題 (ALV 依欄寬挑用)
outputlen	顯示寬度

fieldcat 欄位	用途
key	'X' = 鍵欄位 (固定不捲動、上色)
do_sum	'X' = 數值欄自動加總
no_out	'X' = 先隱藏 (使用者可自行叫出)
currency / cfieldname	金額欄位的幣別 (固定值 / 參考欄位)

若資料內表直接用 DDIC 結構 (如 `TYPE TABLE OF scarr`) , 也可以呼叫 `REUSE_ALV_FIELDATALOG_MERGE` 自動產生 fieldcat ; 自訂結構則手工建——兩種都會遇到。

3. MACRO : 把重複程式碼縮成一行

上面「加一欄」要五行, 十個欄位就是五十行。傳統程式用 **MACRO** 壓縮這種重複:

```

DEFINE mc_add_field.
  CLEAR gs_fieldcat.
  gs_fieldcat-fieldname = &1.      " &1~&9 是佔位參數, 呼叫時依順序代入
  gs_fieldcat-seltext_1 = &2.
  gs_fieldcat-outputlen = &3.
  APPEND gs_fieldcat TO gt_fieldcat.
END-OF-DEFINITION.

* 呼叫: 像自創的指令一樣用 ( 注意: 參數用空白隔開, 不是逗號 )
mc_add_field 'CARRID'   '航空公司代碼' 12.
mc_add_field 'CARRNAME' '航空公司名稱' 24.
mc_add_field 'CURRCODE' '幣別'         6.

```

MACRO 的本質是編譯前的文字展開 (把 &n 換成實參後原地貼上) , 因此:

- 不能下中斷點、除錯時單步直接跳過整個 MACRO——出錯很難查。
- 只在定義它的程式內有效, 必須先定義後使用。
- 新程式碼不要寫 **MACRO** (用 FORM / Method 取代) , 但舊程式極常見 (本專案 ZDQM0001F01 的 `gui_download` 就是) , 必須看得懂。

4. 呼叫 REUSE_ALV_GRID_DISPLAY

```

CALL FUNCTION 'REUSE_ALV_GRID_DISPLAY'
  EXPORTING
    i_callback_program = sy-repid      " 回呼程式: 本程式自己
    it_fieldcat         = gt_fieldcat
  TABLES
    t_outtab           = gt_carriers    " 要顯示的資料內表
  EXCEPTIONS
    program_error      = 1
    OTHERS              = 2.

```

```
IF sy-subrc <> 0.
  WRITE / 'ALV 顯示失敗'.
ENDIF.
```

- `i_callback_program` 給 `sy-repid` (目前程式名) , ALV 事件回呼 (如自訂頁首、雙擊) 都靠它找回你的程式。
- CALL FUNCTION 的完整語法與 EXCEPTIONS 機制 , 講義 15 詳解 ; 此處先照樣板使用並檢查 `sy-subrc` 。

4.1 layout : 整體版面

```
DATA gs_layout TYPE slis_layout_alv.
gs_layout-zebra          = 'X'.      " 斑馬紋 (隔行底色)
gs_layout-colwidth_optimize = 'X'.    " 欄寬自動依內容調整

CALL FUNCTION 'REUSE_ALV_GRID_DISPLAY'
EXPORTING
  i_callback_program = sy-repid
  is_layout          = gs_layout      " 多傳這個參數
  it_fieldcat        = gt_fieldcat
TABLES
  t_outtab           = gt_carriers
EXCEPTIONS
  program_error      = 1
  OTHERS              = 2.
```

5. 完整流程回顧

1. SELECT 資料進內表 (講義 6)
2. 建 fieldcat (手工 + MACRO , 或 MERGE 自動產生)
3. (可選) 設 layout
4. CALL `REUSE_ALV_GRID_DISPLAY` , 檢查 `sy-subrc`

6. 常見錯誤與陷阱

症狀	原因
某欄位就是不顯示	<code>fieldname</code> 沒大寫或拼錯 (不會報錯 , 默默消失)
整個 ALV 空白	fieldcat 是空的 , 或 <code>t_outtab</code> 傳錯內表
金額欄小數位不對	沒設 <code>currency</code> / <code>cfieldname</code>
MACRO 呼叫報語法錯誤	參數用了逗號 (要用空白) , 或呼叫寫在定義之前
除錯進不去某段邏輯	那段是 MACRO——換成在展開前後下斷點觀察變數

8. 進階篇 : REUSE_ALV_GRID_DISPLAY_LVC——可編輯 ALV

前七節的 ALV 是唯讀展示；實務上大量的作業型報表長這樣：清單前方有選取 checkbox、部分欄位可直接編輯、上方有自訂按鈕（如「確認」），按下去處理選取列、處理完的列反灰鎖定不能再改。這要換用 LVC 版本：

	REUSE_ALV_GRID_DISPLAY	REUSE_ALV_GRID_DISPLAY_LVC
fieldcat 型別	slis_t_fieldcat_alv (參數 it_fieldcat)	lvc_t_fcat (參數 it_fieldcat_lvc)
layout 型別	slis_layout_alv (is_layout)	lvc_s_layo (is_layout_lvc)
欄位標題	seltext_l	coltext
可編輯 / 樣式 / grid 事件	幾乎不支援	完整支援

LVC 型別是底層 OO 元件 cl_gui_alv_grid 的原生型別，_LVC 版 FM 只是幫它包了全螢幕外殼——所以 grid 的能力（編輯、樣式、事件）都拿得到。維護舊程式兩種都會遇到；要互動就選 _LVC 版。

8.1 可編輯欄位與選取 checkbox

fieldcat 多了兩個關鍵欄位：

```
DATA: gt_fieldcat TYPE lvc_t_fcat,  
      gs_fieldcat TYPE lvc_s_fcat.  
  
gs_fieldcat-fieldname = 'SEL'.  
gs_fieldcat-coltext   = '選'.  
gs_fieldcat-checkbox  = 'X'.      " 顯示成 checkbox  
gs_fieldcat-edit      = 'X'.      " 可以點 ( 沒有 edit 的 checkbox 只能看 )  
APPEND gs_fieldcat TO gt_fieldcat.  
  
gs_fieldcat-fieldname = 'REMARK'.  
gs_fieldcat-coltext   = '備註'.  
gs_fieldcat-edit      = 'X'.      " 一般可編輯欄位
```

對應的資料結構要自己加 sel TYPE c LENGTH 1 這類欄位。重要觀念：使用者在格子裡打的字不會即時回到 internal table——_LVC 全螢幕 wrapper 會在呼叫 i_callback_user_command 之前自動同步好資料，所以 USER_COMMAND 回呼裡讀到的 gt_data 已經是最新值，不用自己做任何事（8.4 會說明這跟 OO 版 cl_gui_alv_grid 的差異）。

8.2 STYLEFNAME：列級樣式——Fcode 執行後把 checkbox 反灰

需求：按「確認」處理完的列，checkbox 與備註欄要反灰（disabled），不能再改。做法是給每一列帶一張「樣式表」：

```
TYPES: BEGIN OF ty_row,  
       sel      TYPE c LENGTH 1,  
       carrid   TYPE scarr-carrid,
```

```

...
    celltab TYPE lvc_t_styl,      " 這一系列的「欄位樣式表」
END OF ty_row.

```

```

DATA gs_layout TYPE lvc_s_layo.
gs_layout-stylefname = 'CELLTAB'.      " ★ 告訴 ALV：樣式放在哪個欄位

```

之後 (通常在自訂 Fcode 的處理邏輯裡) 往該列的 **celltab** 塞「哪個欄位、什麼樣式」：

```

DATA ls_style TYPE lvc_s_styl.
FIELD-SYMBOLS <ls_row> TYPE ty_row.      " 指向內表的一列 ( 講義 16 )

LOOP AT gt_data ASSIGNING <ls_row> WHERE sel = 'X'.
    <ls_row>-status = '已確認'.
    CLEAR <ls_row>-celltab.
    ls_style-fieldname = 'SEL'.
    ls_style-style      = cl_gui_alv_grid=>mc_style_disabled.    " 反灰
    INSERT ls_style INTO TABLE <ls_row>-celltab.
    ls_style-fieldname = 'REMARK'.
    INSERT ls_style INTO TABLE <ls_row>-celltab.
ENDLOOP.

```

補充： **ASSIGNING <ls_row>** 也可以寫成行內宣告 **ASSIGNING FIELD-SYMBOL(<ls_row>)** (7.40 之後的新寫法，省掉上面 **FIELD-SYMBOLS** 那一行)，講義 16 補充、講義 26 詳述。

刷新畫面後 (**selfield-refresh = 'X'**，見 8.3)，這些列的 checkbox / 備註就是灰的。常用樣式常數還有 **mc_style_enabled** (解鎖)、**mc_style_hotspot**。**celltab** 欄位本身不要放進 fieldcat。

8.3 自訂按鈕：PF-STATUS 與 USER_COMMAND 回呼

自訂 Fcode (如 **ZCONF** 「確認」按鈕) 要兩個回呼：

```

CALL FUNCTION 'REUSE_ALV_GRID_DISPLAY_LVC'
EXPORTING
    i_callback_program      = sy-repid
    i_callback_pf_status_set = 'SET_PF_STATUS'    " 掛自訂工具列
    i_callback_user_command = 'USER_COMMAND'      " 接使用者動作
    is_layout_lvc          = gs_layout
    it_fieldcat_lvc        = gt_fieldcat
    i_grid_settings        = gs_glay              " 見 8.4
TABLES
    t_outtab               = gt_data ...

```

```

FORM set_pf_status USING pt_extab TYPE slis_t_extab.
    SET PF-STATUS 'STANDARD' EXCLUDING pt_extab.
ENDFORM.

```

```

FORM user_command USING pv_ucomm      TYPE sy-ucomm
                        ps_selfield TYPE slis_selfield.

CASE pv_ucomm.
  WHEN 'ZCONF'.
    PERFORM confirm_selected.          " 8.2 的反灰邏輯在這裡
    ps_selfield-refresh = 'X'.         " 讓 ALV 重讀內表 ( 樣式才會生效 )
  ENDCASE.
ENDFORM.

```

- GUI Status **STANDARD** 的標準做法：SE80 從程式 **SAPLKKBL** 複製 Status **STANDARD_FULLSCREEN** 到自己的程式，再把自訂按鈕 (Fcode **ZCONF**) 加進 Application Toolbar——這樣標準功能 (排序、篩選、匯出) 全數保留。
- **user_command** 的兩個參數是固定介面：**sy-ucomm** (哪個 Fcode) 與 **slis_selfield** (游標所在列/欄、**refresh** 開關)。

8.4 EDT_CLL_CB 與 DATA_CHANGED——「離開已編輯儲存格」的回呼

可編輯 ALV 的痛點：使用者改了格子、**沒按 Enter** 就去按別的東西，程式看到的還是舊值；想「一離開儲存格就檢核 / 連動計算」，預設根本不觸發。跟前面所有回呼 (PF-STATUS、USER_COMMAND) 同一套模式：**開關設定 + 事件表掛 FORM**，全程不需要碰底層 grid 物件，也不需要 class / SET HANDLER (那是 **cl_gui_alv_grid** 的 OO 用法，留待 OOP 課)。

其一：**I_GRID_SETTINGS-EDT_CLL_CB = 'X'**

```

DATA gs_glay TYPE lvc_s_glay.
gs_glay-edt_cll_cb = 'X'.      " ★ 離開已編輯儲存格 → 觸發 DATA_CHANGED

```

其二：**IT_EVENTS** 掛 **SLIS_EV_DATA_CHANGED** 事件，指到自己的 **FORM**

```

DATA: gt_events TYPE slis_t_event,
      gs_event  TYPE slis_alv_event.          " 事件表的一列

gs_event-name = slis_ev_data_changed.        " 哪個事件
gs_event-form = 'DATA_CHANGED'.              " 事件發生時回呼哪個 FORM
APPEND gs_event TO gt_events.

CALL FUNCTION 'REUSE_ALV_GRID_DISPLAY_LVC'
  EXPORTING
    ...
    i_grid_settings = gs_glay                " 其一
    it_events       = gt_events              " 其二
  TABLES
    t_outtab       = gt_data ...

```

補充：VALUE #(...) 新寫法：事件表那三行（填 `gs_event + APPEND`）在 7.40 之後可以寫成一句 `APPEND VALUE #( name = slis_ev_data_changed form = 'DATA_CHANGED' ) TO gt_events.`，同時建出一列並 `APPEND`，型別由 `gt_events` 的列自動推導，效果相同；`VALUE` 的完整用法留到講義 26。

其三：FORM 介面固定，第一個參數收 `cl_alv_changed_data_protocol` 物件（型別是固定的，不是自己宣告的 class）：

```
FORM data_changed USING pr_data_changed TYPE REF TO cl_alv_changed_data_protocol.
  DATA ls_cell TYPE lvc_s_modi.

* mt_good_cells：這次「改了哪些格子」（列號/欄名/新值）
  LOOP AT pr_data_changed->mt_good_cells INTO ls_cell.
    IF ls_cell-fieldname = 'REMARK' AND ls_cell-value CS '!'.
      CALL METHOD pr_data_changed->add_protocol_entry " 檢核錯誤：ALV 彈出錯誤紀錄
        EXPORTING
          i_msgid      = '0K'
          i_msgty      = 'E'
          i_msgno      = '000'
          i_msgv1      = '備註不可含驚嘆號'
          i_fieldname  = ls_cell-fieldname
          i_row_id     = ls_cell-row_id.
    ENDIF.
  ENDLOOP.
ENDFORM.
```

跟 8.3 的 `set_pf_status / user_command` 完全同一套骨架：EXPORTING 參數或 `IT_EVENTS` 告訴 wrapper 「發生 X 時回呼哪個 FORM」，FORM 介面固定、直接照抄簽名。`EDT_CLL_CB` 是控制觸發時機的開關（離開儲存格 vs 只有按 Enter）；`IT_EVENTS` 是控制接不接得到這個事件（沒掛 FORM，開關開了也沒人處理）。

另一個相關招式：`USER_COMMAND` 開頭呼叫 `check_changed_data( )` 可以把畫面上還沒處理的編輯先刷回內表——但這是 OO 版 `cl_gui_alv_grid` 的方法，`_LVC` 全螢幕版不需要：wrapper 在呼叫 `i_callback_user_command` 之前已經自動同步好資料，直接在 `user_command` 裡讀 `gt_data` 就是最新值。

9. 常見錯誤與陷阱（進階篇）

症狀	原因
checkbox 顯示出來但點不動	fieldcat 只給 <code>checkbox = 'X'</code> 沒給 <code>edit = 'X'</code>
勾了選取、按自訂按鈕卻讀不到勾選	通常是 fieldcat 的 <code>checkbox / edit</code> 沒同時設定——確認資料真的寫回了內表（見 8.1）
離開儲存格 <code>DATA_CHANGED</code> 都不觸發	<code>i_grid_settings-edt_cll_cb</code> 沒設 'X'，或 <code>it_events</code> 沒掛 <code>slis_ev_data_changed</code> 對應的 FORM（開關開了沒人接、掛了 FORM 沒開關，兩個條件都要滿足）

症狀	原因
反灰完全沒效果	layout 沒設 <code>stylefname</code> 、欄名沒大寫、或 Fcode 處理後忘了 <code>selfield-refresh = 'X'</code>
執行就 dump (狀態不存在)	<code>SET PF-STATUS 'STANDARD'</code> 但 GUI Status 還沒複製建立
樣式欄位出現在畫面上	<code>CELLTAB</code> 誤放進 fieldcat——它是給 ALV 看的，不是給人看的
fieldcat 欄位對不上	LVC 的標題欄位是 <code>coltext</code> ，不是 slis 的 <code>seltext_1</code>

10. 課堂練習

- 基礎：完成 [ex09](#)——SELECT SCARR，用 MACRO 建 fieldcat，以 REUSE_ALV_GRID_DISPLAY 顯示，試玩排序 / 篩選 / 匯出。
- 進階：完成 [ex24](#)——改用 REUSE_ALV_GRID_DISPLAY_LVC 做「可勾選 + 可編輯 + 確認後反灰」的作業型 ALV，並用 EDT_CLL_CB 體驗離開儲存格即時檢核。